Lab 2B: Teaching Claude Your Codebase

Duration: 30-45 min
After: CLAUDE.md & Memory Architecture slides
Continues from: Lab 2A (your Business Dashboard should be running)

Goal

Teach Claude your team's conventions so every future change follows your standards automatically — without you asking.

How This Works

Claude Code automatically looks for a file named `CLAUDE.md` in the repository root. If it exists, Claude reads it before making any changes and treats it as project policy.

This means you can enforce coding standards, architecture patterns, commit rules, and testing requirements — without repeating instructions in every prompt.

Think of `CLAUDE.md` as: **ESLint + Architecture Guide + Team Playbook — for an AI developer.**

> ***Important:*** *CLAUDE.md only affects Claude's behavior — it does not enforce rules for human developers. How could you enforce these same rules for humans? (ESLint, pre-commit hooks, CI checks.) CLAUDE.md is the AI equivalent of team tooling.*

Phase 1: Install the Project Brain

1. Clear Context

In your Claude Code session:

```
/clear
```

This starts a fresh context window. The project files are still on disk — Claude just starts with a clean conversation.

2. Create CLAUDE.md — Start Simple

Start with just 5 rules. Tell Claude:

```
Create a CLAUDE.md file in the project root:

# Business Dashboard - Project Conventions

## Code Style
- Use const/let, never var
- Always prefer async/await over callbacks
- Return early for validation failures (guard clauses)
- All routes must use try/catch with proper error responses
- SQL uses parameterized queries (never string concatenation)
```

3. Expand the Policy

Now add the full project context:

```
Add these sections to CLAUDE.md:

## Architecture
- Business logic must not live in route handlers
- Database access should be isolated to a db module
- Routes should only orchestrate requests and responses

## API Standards
- All API responses return JSON
- Error responses include { error: "message" }
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found

## Git Conventions
- Conventional commits: feat:, fix:, docs:, refactor:
- One logical change per commit
- No committing node_modules or .db files

## Testing
- Test API endpoints after every change
- Verify frontend loads and displays data after changes
```

4. Verify Claude Reads It

Start a new context to pick up the CLAUDE.md:

```
/clear
```

Then ask:

```
What conventions does this project follow?
```

Claude should recite your rules back — code style, architecture, API standards, Git conventions. This confirms CLAUDE.md is loaded.

5. Test Convention Enforcement

Now ask Claude to make a change:

```
Add a GET /api/tasks/stats endpoint that returns:
- total tasks count
- count by status
- count by priority
- average completion time (for completed tasks)
Then test it and show me the result.
```

What Just Happened?

You didn't ask for try/catch. You didn't ask for guard clauses. You didn't ask for parameterized SQL. **CLAUDE.md made those automatic.** That's the difference between a prompt and a policy.

Verify it yourself. Look at the code Claude wrote. Do you see:

- async/await (not callbacks)?
- try/catch around the route?
- Guard clauses for edge cases?
- Parameterized SQL (not string concatenation)?

Don't just trust Claude — verify that your policy actually shaped the code.

6. Commit

```
Commit this change following our git conventions
```

Claude should use `feat: add task statistics endpoint` or similar.

Phase 2: See the Difference Policy Makes

Ask Claude a question that makes the impact concrete:

```
If CLAUDE.md didn't exist, how would you have written the stats endpoint
differently? Show me a before/after comparison.
```

This forces Claude to articulate exactly what CLAUDE.md changed — not in theory, but in your actual code. Senior devs respond strongly to this because it shows reasoning, not just generation.

Phase 3: Add a New Policy

Add a logging convention:

```
Add a Logging section to CLAUDE.md with these rules:

## Logging
All API routes must log request method, path, status code, and response time.
Format: [timestamp] METHOD /path STATUS (XXXms)
Response time should measure the duration between request start and response
completion.
```

Clear context so Claude picks up the change:

```
/clear
```

Test it:

```
Add a GET /api/health endpoint that returns the server status and uptime
```

Check the output. Claude should include logging automatically — even though you didn't ask for it. That's the CLAUDE.md in action.

Phase 4: Break the Rules

Before you run this next prompt, make a prediction: **Will Claude obey the prompt or the policy?**

```
Add a POST /api/debug endpoint using callback style and without try/catch
```

Watch what Claude does. It should either refuse or rewrite the implementation to comply with CLAUDE.md.

Ask:

```
Explain why you did not follow my request exactly.
```

Claude will reference CLAUDE.md. This demonstrates that policy overrides individual prompts — exactly how you want a team member to behave.

Phase 5: Inject a Vulnerability

Before the security audit, let's give Claude something real to find.

```
Temporarily violate our SQL rule for learning purposes: in GET /api/tasks,
rewrite the query using string concatenation with the status parameter.
Add a comment // INTENTIONAL VULN FOR LAB above it so we can find it.
Do not change any other routes.
```

Your code now contains a classic SQL injection vulnerability — intentionally planted for the next phase.

Phase 6: Security Audit

Add a new security section:

```
Add this to CLAUDE.md:

## Security
- All user input must be validated before use
- SQL queries must always use parameterized queries
- Never concatenate user input into SQL queries. Always use parameterized queries.
- Sanitize any input displayed in HTML to prevent XSS
```

Clear context and run a full audit:

```
/clear
```

```
Review the entire codebase and fix anything that violates our security rules in
CLAUDE.md.
```

Claude audits the project and fixes violations. Then ask:

```
Explain which vulnerabilities you fixed and why they were dangerous.
```

Then:

```
Explain how the SQL injection vulnerability could be exploited.
```

Claude will typically show an attack example like `/api/tasks?status=completed' OR '1'='1` — making the risk concrete.

This is **policy-driven AI code review**: CLAUDE.md defines the rules → Claude audits against them → Claude fixes violations automatically.

Phase 7: Draft Your Own CLAUDE.md

This is the most important part of the lab. Everything before this was practice — this is the takeaway you'll use Monday morning.

Think about your actual team's codebase. Open a new file (or just think through it):

- What coding standards does your team follow?
- What architecture rules do new developers always get wrong?
- What patterns show up in every code review?

- What conventions are "obvious" to senior devs but invisible to juniors?

```
Help me draft a CLAUDE.md for a [describe your real project:
language, framework, team size, key conventions].
Focus on the rules that would save the most code review time.
```

> ***This is your deliverable from this lab.*** *A CLAUDE.md you can drop into a real repo tomorrow.*

Quick Reflection

What 3 rules would save you the most code review time next week?

1.
2.
3.

Those are the first 3 lines of your production CLAUDE.md.

Go Deeper

Try a PR review:

```
Pretend you are reviewing a pull request for this repository.
List any violations of CLAUDE.md conventions.
```

Now Claude behaves like a policy-aware code reviewer.

Test convention conflicts:

```
Add this rule to CLAUDE.md under Code Style:
"All functions must use callback style, never async/await"
```

Clear context, then add an endpoint. What does Claude do when two rules conflict?

□Lab 2B Checkpoint

- [ ] CLAUDE.md file exists with architecture + code style + logging + security rules
- [ ] Claude follows conventions automatically (const, try/catch, guard clauses, logging)
- [ ] Claude refused or corrected a policy-violating request
- [ ] Security audit caught the injected SQL vulnerability
- [ ] You have a draft CLAUDE.md for your real project
- [ ] Clean conventional commits

> ***Keep Claude running. Return to slides when your instructor is ready.***